Operations Optimisation Assignment

Reproducing the MILP for the Drone-Assisted Pickup and Delivery Problem

Mulumba & Diabat, *Transp. Res. E* 181 (2024) 103377

Rohit Roy Chowdhury, Swayam Kuckreja (5726549), Kai Murtagh (5685095)
TU Delft Aerospace Engineering

July 2026

1 Introduction

This report reproduces the mixed-integer linear program of Mulumba and Diabat [1] for the Drone-Assisted Pickup and Delivery Problem (DAPDP). A truck performs a sequence of paired pickup and delivery requests while a single on-board drone may peel off, deliver a light parcel, and rendezvous with the truck downstream. The goal is to minimise total fuel cost subject to truck capacity, drone payload and endurance, and a maximum route duration.

The contributions of this work are:

1. A clean Python/Gurobi implementation of all 37 constraints of the paper (Section 3.2 of the original) plus two of the three valid-inequality families of Section 3.3: the cost-bound epigraph (Prop. 1) and the knapsack cuts (43)–(44). The symmetry-breaking cut (41)–(42) is omitted because it is nonlinear (a product $u_i^v\, x_{0i}^v$) and so not directly expressible in the MILP.
2. A truck-only baseline obtained by restricting the same model with $C'{=}\emptyset$, used to compute the saving $SPDP = 1 - z_{\mathrm{DAPDP}}/z_{\mathrm{PDP}}$.
3. A verification suite of seven independently-audited test scenarios covering each major class of constraint.
4. A sensitivity analysis on drone endurance, drone speed and the drone cost factor α.
5. A small-scale validation on the paper's instance sizes $n \in \{6, 8, 10\}$, which reproduces the qualitative finding that the drone yields positive cost savings. The mean SPDP across these small cells is $\approx 14\%$; the paper's headline $\approx 25\%$ is an average over its *large*-scale set (up to $n{=}150$), and our smaller, grid-dependent savings are consistent with the paper's own small-instance rows (Table A.6).

The paper contains a small number of typographic ambiguities (constraint (28), the demand sign convention in §5.2, and the launch-side treatment in constraints (10) and (26)) which we resolve and document explicitly in §3.

Reproducibility. A single shell script `run_all.sh` regenerates every CSV, PDF figure and the report itself from scratch. All numerical values reported below come directly from the most recent run of that script.

2 Problem statement

Let $N^+ = \{1, \ldots, 2n+1\}$, $N^0 = \{0, \ldots, 2n\}$, $C = \{1, \ldots, 2n\}$ be the customer node set, partitioned into pickups $\Phi^+ = \{1, \ldots, n\}$ and deliveries $\Phi^- = \{n{+}1, \ldots, 2n\}$; delivery node $n{+}i$ is paired with pickup i. Node 0 is the start depot, node $2n{+}1$ a virtual end depot at the same

location. The drone-eligible subset $C' \subseteq \Phi^-$ contains those deliveries whose demand $\le Q'$, the drone payload limit. Truck travel time is $\tau_{ij} = d_{ij}/s$, drone travel time $\tau'_{ij} = d_{ij}/s'$ with d_{ij} the Euclidean distance.

Demand sign convention. We use $q_i > 0$ at pickups and $q_i < 0$ at deliveries. This is the *opposite* of the textual statement in the paper's §5.2 but is the only convention consistent with the literal reading of the weight balance (28), where positive contributions of q_j must increase the truck load on arrival.

Decision variables.

- $x^v_{ij} \in \{0,1\}$: truck v traverses arc (i,j).
- $y^v_{ijk} \in \{0,1\}$: a drone-sortie launch i, drone delivery j, rendezvous k for truck v.
- $p^v_{ij} \in \{0,1\}$: i precedes j in truck v's route.
- $t^v_i,\ t'^v_i \ge 0$: truck and drone arrival times at i.
- $u^v_i \in \{1,\dots,|N|\}$: position of i in truck v's sequence.
- $w^v_i \ge 0$: truck weight after visiting i.
- $\eta \ge 0$: epigraph variable; objective $\min \eta$ with $\eta \ge \sum_{v,i,j} c_{ij} x^v_{ij} + \sum_{v,(i,j,k)\in P} (c'_{ij}+c'_{jk}) y^v_{ijk}$.

Mathematical model. The implemented MILP is given below in the paper's equation numbering, with the deviations of §3 already folded in (the $-M$ sign in (28), the epigraph objective (38)–(39), the depot-launch ban in (26), and the elision of (10) and (35)). Two big-M constants are used: $M_{\text{pos}} = 2n{+}2$ in the position constraints and $M_t = T_{\max} + 10\,\text{h}$ in the time constraints. The drone sortie set is $\mathcal{P} = \{(i,j,k) : i \ne 0,\ j \in C',\ k \in N^+,\ i,j,k \text{ distinct}\}$, pre-filtered for endurance.

$$\min\ \eta \tag{38}$$

$$\eta \ge \sum_{v\in\mathcal{V}} \Big(\sum_{(i,j)\in\mathcal{A}} c_{ij} x^v_{ij} + \sum_{i\in N^0}\sum_{j\in C}\sum_{k\in N^+} (c'_{ij}+c'_{jk})\, y^v_{ijk} \Big) \tag{39}$$

$$\sum_v \Big(\sum_{i\in N^0: i\ne j} x^v_{ij} + \sum_{i\in N^0: i\ne j}\sum_{k\in N^+} y^v_{ijk} \Big) = 1, \quad \forall j \in C \tag{2}$$

$$\sum_{j\in N^+} x^v_{0j} \le 1, \quad \sum_{i\in N^0} x^v_{i,2n+1} \le 1, \qquad \forall v \tag{3,4}$$

$$1 + u^v_i - u^v_j \le (1 - x^v_{ij})\, M_{\text{pos}}, \quad \forall i \in C,\ j \in N^+\!\setminus\!\{i\},\ v \tag{5}$$

$$\sum_{i\in N^0: i\ne j} x^v_{ij} = \sum_{k\in N^+: k\ne j} x^v_{jk}, \quad \forall j \in C,\ v \tag{6}$$

$$\sum_{j\in C: j\ne i}\sum_{k\in N^+} y^v_{ijk} \le 1,\ \ \forall i \in N^0; \qquad \sum_{i\in N^0: i\ne k}\sum_{j\in C} y^v_{ijk} \le 1,\ \ \forall k \in N^+ \tag{7,8}$$

$$2y^v_{ijk} \le \textstyle\sum_{h\in N^0: h\ne i} x^v_{hi} + \sum_{l\in N^0: l\ne k} x^v_{lk}, \quad \forall (i,j,k) \in \mathcal{P},\ v \ \ (\text{launch term} \equiv 1 \text{ if } i{=}0) \tag{9}$$

$$u^v_k - u^v_i \ge 1 - M_{\text{pos}}\big(1 - \textstyle\sum_j y^v_{ijk}\big), \quad \forall i \in C,\ k \in N^+\!\setminus\!\{i\},\ v \tag{11}$$

$$t'^v_i \ge t^v_i - M_t\big(1 - \textstyle\sum_{j,k} y^v_{ijk}\big), \quad t'^v_i \le t^v_i + M_t\big(1 - \textstyle\sum_{j,k} y^v_{ijk}\big), \ \ \forall i \in C,\ v \tag{12,13}$$

$$t'^v_k \ge t^v_k - M_t\big(1 - \textstyle\sum_{i,j} y^v_{ijk}\big), \quad t'^v_k \le t^v_k + M_t\big(1 - \textstyle\sum_{i,j} y^v_{ijk}\big), \ \ \forall k \in C,\ v \tag{14,15}$$

$$t^v_k \ge t^v_h + \tau_{hk} + d_h + s_L \textstyle\sum_{l,m} y^v_{klm} + s_R \sum_{i,j} y^v_{ijk} - M_t(1 - x^v_{hk}), \ \ \forall h \in N^0,\ k \in N^+\!\setminus\!\{h\},\ v \tag{16}$$

$$t'^v_j \ge t'^v_i + \tau'_{ij} - M_t\big(1 - \textstyle\sum_k y^v_{ijk}\big), \quad \forall j \in C',\ i \in N^0\!\setminus\!\{j\},\ v \tag{17}$$

$$t'^v_k \ge t'^v_j + \tau'_{jk} + d'_j + s_R - M_t\big(1 - \textstyle\sum_i y^v_{ijk}\big), \quad \forall j \in C',\ k \in N^+\!\setminus\!\{j\},\ v \tag{18}$$

$$t'^v_k - (t'^v_j - \tau'_{ij}) \le e + M_t(1 - y^v_{ijk}), \quad \forall (i,j,k) \in \mathcal{P},\ v \tag{19}$$

$$u^v_i - u^v_j \ge 1 - M_{\text{pos}}\, p^v_{ij}, \quad u^v_i - u^v_j \le -1 + (1 - p^v_{ij})\, M_{\text{pos}}, \ \ \forall i,j \in C : i \ne j,\ v \tag{20,21}$$

$$p^v_{ij} + p^v_{ji} = 1, \quad \forall i,j \in C : i < j,\ v \tag{22}$$

$$t'^v_l \ge t'^v_k - M_t\big(3 - \textstyle\sum_j y^v_{ijk} - \sum_{m,n} y^v_{lmn} - p^v_{il}\big), \ \ \forall i \in N^0,\ k \in N^+\!\setminus\!\{i\},\ l \in C\!\setminus\!\{i,k\},\ v \tag{23}$$

$$x^v_{0,2n+1} = 0, \quad \forall v \tag{24}$$

Table 1: Constraint groups of the DAPDP MILP and their implementation.

Eqs.	Purpose	Implementation note
(2)	Each customer served exactly once	—
(3)–(4)	Depot start/end at most once per truck	—
(5)	Sub-tour elimination (MTZ on u)	big-$M_{\text{pos}} = 2n + 2$
(6)	Truck flow conservation	—
(7)–(8)	At most one launch/return per node	—
(9)–(10)	Launch and rendezvous on truck route	(10) subsumed by (9), see §3
(11)	Launch precedes rendezvous in u	—
(12)–(15)	Time synchronisation of truck and drone	big-$M_t = T_{\max} + 10\,\text{h}$
(16)	Truck-arrival recursion (with launch/recovery)	—
(17)–(18)	Drone-arrival recursion	—
(19)	Drone endurance $\le e$	also pre-filtered into P
(20)–(22)	Consistency of u and p	—
(23)	No second sortie before return of first	—
(24)	Forbid empty depot-to-depot trip	—
(25)	Same vehicle for paired (i, $i+n$)	—
(26)	Pickup before launch when drone delivers j	extended to i=0, see §3
(27)	Pickup precedes delivery on truck	—
(28)	Truck weight evolution	sign correction, see §3
(31)–(34)	Time bounds, $T_{\max}$	—
(35)	$p^v_{0j} = 1$	omitted; see §3
(36)–(37)	Bounds on u, w	encoded in variable bounds
(43)–(44)	Knapsack valid inequalities (Section 3.3)	on; symmetry cut (41) omitted (nonlinear)

$$\sum_{j\in N^+: j\neq i} x^v_{ij} = \sum_{j\in N^+: j\neq n+i} x^v_{n+i,j} + \sum_{l\in N^0}\sum_{k\in N^+} y^v_{l,n+i,k}, \quad \forall i \in \Phi^+,\, v \tag{25}$$

$$u^v_i - u^v_{j-n} \ge 1 - M_{\text{pos}}(1 - y^v_{ijk}), \;\; \forall (i,j,k) \in \mathcal{P},\, j \in \Phi^-; \qquad y^v_{0jk} = 0 \tag{26}$$

$$t^v_{n+i} \ge t^v_i + \tau_{i,n+i} + d_i, \quad \forall i \in \Phi^+,\, v \tag{27}$$

$$w^v_j \ge w^v_i + q_j + \sum_{(j,m,k)\in\mathcal{P}} q_m\, y^v_{jmk} - Q(1 - x^v_{ij}), \;\; \forall i \in N^0,\, j \in C : i \neq j,\, v \tag{28}$$

$$t^v_0 = {t'}^v_0 = 0; \quad t^v_{2n+1} \le T_{\max}, \;\; {t'}^v_{2n+1} \le T_{\max}, \quad \forall v \tag{31–34}$$

$$1 \le u^v_i \le 2n{+}2, \quad 0 \le w^v_i \le Q \tag{36,37}$$

$$x^v_{ij} + \textstyle\sum_{k\in N^+} y^v_{ijk} \le 1, \quad \forall i \in N^0,\, j \in C' : i \neq j,\, v \tag{43}$$

$$x^v_{ij} + x^v_{jk} + x^v_{ik} + y^v_{ijk} \le 2, \quad \forall (i,j,k) \in \mathcal{P},\, v \tag{44}$$

Variables are binary (x, y, p) or non-negative continuous (t, t', w, η), with u integer. Table 1 summarises the role of each group; the non-obvious implementation choices are detailed next.

3 Interpretation decisions where the paper is ambiguous

The paper has a few places where the printed model is either mathematically inconsistent or requires assumptions that are not stated. Implementing it verbatim would yield an infeasible or trivially-relaxed program, so we deviate as follows.

(28) Truck weight balance — big-M sign. The paper writes

$$w^v_j \ge w^v_i + q_j + \sum_{(i,j,k)\in P} q_m\, y^v_{jmk} + Q\,(1 - x^v_{ij}),$$

which has the wrong sign of the big-M. With $+M$, every arc with $x^v_{ij} = 0$ would force $w^v_j \le w^v_i + q_j + \cdots - Q$ (re-arranging), which is structurally infeasible whenever Q is large. The standard relaxation is $-M(1 - x^v_{ij})$, which deactivates the inequality whenever the arc is unused. We implement this minus sign.

Demand-sign convention. The paper's §5.2 says "demand q_i is randomly drawn from $U(0, 2.27)$ with probability 0.86 ... and the pickup demand is $-q_{n+i}$." Taking this literally gives $q^+ < 0$ at pickups and $q^- > 0$ at deliveries, which contradicts (28) (truck load should *rise* on a pickup). We adopt the sign convention required by (28) and described in our `instance_generator.py`: $q_i > 0$ at pickup, $q_i < 0$ at delivery.

(26) Pickup-before-launch: depot launches. Constraint (26) reads $u_i^v - u_{j-n}^v \geq 1 - M(1 - y_{ijk}^v)$ for $j \in \Phi^-$, $i \in C$. Two issues arise. First, i runs over C only, so $i = 0$ is implicitly excluded; nothing in the paper prevents launching a delivery sortie directly from the depot, which would let the drone deliver before the truck has performed the corresponding pickup. Second, when $i \in C$ but the corresponding pickup $j - n$ has not been visited yet, u_{j-n}^v is undefined relative to u_i^v. We strengthen (26) by setting $y_{0jk}^v = 0$ for every $j \in \Phi^-$, $k \in N^+$. With one truck this rules out the only physically non-implementable depot-launch case, and matches the paper's heuristic (Algorithm 4) which only considers post-pickup launches.

(10) is subsumed by (9). Constraint (10), $\sum_{i,j} y_{0jk}^v \leq \sum_{l \neq k} x_{lk}^v$, demands the rendezvous k be on the truck route when the launch is from the depot. Constraint (9) already requires that, with the convention $\mathbb{1}\{i \in \text{route}\} = 1$ for $i = 0$. We therefore implement only (9) and document this elision.

(35) is omitted. Constraint (35), $p_{0,j}^v = 1$ for all $j \in C$, requires precedence variables on the depot. We omit it because all our other depot-linked constraints (the MTZ inequality, time/position bounds and (3)–(4)) place the depot first implicitly. Including (35) would require tracking p_{0j}^v throughout, which inflates the variable count by $|C|$ without changing the feasible set.

P **pre-filtering.** We pre-filter the universe of sorties $P = \{(i, j, k) : i \neq 0,\ j \in C',\ k \in N^+,\ i, j, k \text{ distinct}\}$ to those satisfying $s_L + (d_{ij} + d_{jk})/s' + d_j^{\text{drone}} + s_R \leq e$. Sorties that fail this geometric check are forbidden by (19) anyway, but filtering them out shrinks the model by roughly an order of magnitude.

4 Implementation

The Python package is organised as

- `instance_generator.py`: random-instance factory matching §5.2 of the paper (Table 1 defaults), plus a manual constructor used by the verification suite.
- `dapdp_model.py`: the MILP, exposing `solve_dapdp(inst, time_limit, ...)` that returns a `Solution` dataclass.
- `baseline.py`: solves the same MILP with $C' = \emptyset$ to obtain z_{PDP} for SPDP computation.
- `verification.py`: seven scenarios, each producing a matplotlib figure plus an independently audited feasibility report.
- `sensitivity.py`: parameter sweeps with CSV and PDF output.
- `validation.py`: small-scale comparison with $\overline{SPDP}$ values reported in the paper.

The solver is Gurobi 13.0.2 with a TU Delft academic licence. Instance sizes up to n=10 on a 30×30 mile grid solve to optimality in seconds-to-minutes, comparable to the runtimes reported in the paper for the corresponding sizes.

5 Verification

Following the assignment guidelines, we devise instances whose feasible solutions are restricted in a well-understood way and verify the solver respects those restrictions. Each scenario is paired with an independent auditor (`audit()` in `verification.py`) that re-derives feasibility from the returned x and y values without using any internal solver state.

5.1 Scenarios

V1, basic feasibility (n=3).
A randomly generated instance. The auditor checks every constraint listed in Table 1. Result: *passed*, $z^\star = \$0.91$, runtime 0.3 s. Figure 1a.

V2, pickup-precedence (handcrafted).
All demands set to 5 kg ($> Q'$), so $C' = \emptyset$ and the truck must serve every pair. The auditor confirms each delivery follows its pickup in the truck sequence: route $0 \to 1 \to 2 \to 3 \to 6 \to 5 \to 4 \to 7$. Figure 1b.

V3, endurance.
The same instance solved with $e = 60$ min and $e = 5$ min. With generous endurance the solver uses 2 drone sorties ($z^\star = \$4.65$); with the 5 min limit no sortie is feasible ($z^\star = \$5.34$, 0 sorties). Figure 2. Both audits pass.

V4, drone payload.
Three deliveries with demands $\{1, 50, 1\}$ kg. Only deliveries 4 and 6 are in C'. The drone serves exclusively delivery 6; delivery 5 (heavy) is on the truck route. Figure 3a.

V5, no sub-tours (n=6).
Verifies the truck's arc set forms a single connected path $0 \to \cdots \to 2n{+}1$ with no detached cycles. Figure 3b.

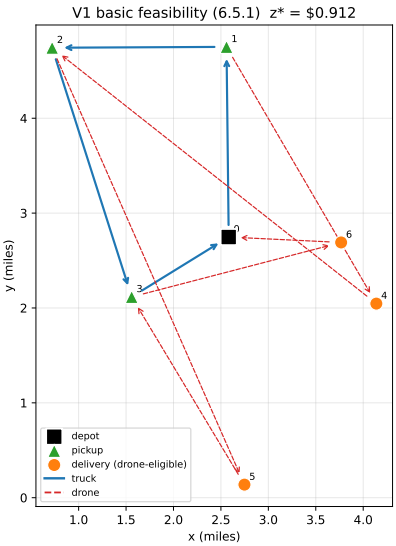
V6, $z_{\mathbf{DAPDP}} \le z_{\mathbf{PDP}}$.
Same instance (n=8, grid 10, seed 3) solved twice. The DAPDP optimum strictly improves on the PDP baseline; the achieved saving is reported in Table 2. Figure 4.

V7, no depot-launch deliveries.
After our extension of (26), $y^v_{0jk} = 0$ for $j \in \Phi^-$. The auditor confirms no drone sortie in the optimum has $i = 0$.

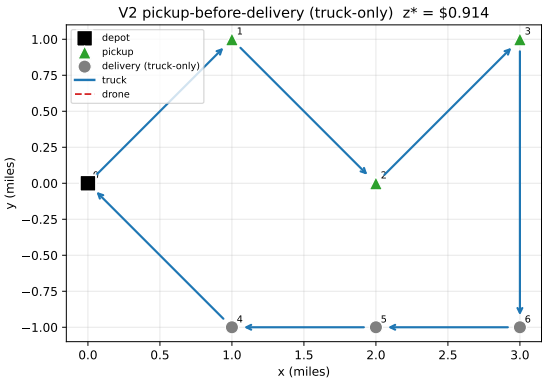
5.2 Audit summary

Table 2: Verification suite results, produced by `verification.py` with the academic licence. For V6 the $z^\star$ column reports the DAPDP optimum; the truck-only baseline is $z_{\mathrm{PDP}} = \$4.316$, giving a saving $S_{\mathrm{PDP}} = 1 - 3.011/4.316 \approx 30\%$.

ID	description	$z^\star$ (\$)	runtime (s)	verdict
V1	basic, n=3	0.912	0.3	passed
V2	pickup-precedence (truck only)	0.914	0.5	passed
V3a	endurance 60 min	4.652	0.6	passed (2 sorties)
V3b	endurance 5 min	5.341	0.6	passed (0 sorties)
V4	drone payload	0.792	0.4	passed (drone serves only {6})
V5	no sub-tours, n=6	4.715	8.0	passed (single path)
V6	DAPDP $\le$ PDP, n=8	3.011	218.0	passed ($S_{\mathrm{PDP}} \approx 30\%$)
V7	no depot-launch deliveries	0.664	0.4	passed (no y_{0jk}=1)



(a) V1 basic feasibility, n=3.



(b) V2 pickup-before-delivery (drone disabled).

Figure 1: Verification scenarios V1 and V2.

6 Sensitivity analysis

We sweep three parameters that the paper highlights as most influential. The ensemble is 12 instances ($n \in \{6, 8\}$, grid sizes $\{10, 20\}$ miles, seeds $\{1, 2, 3\}$); z_{PDP} is computed once per instance and cached, so each sweep value is one DAPDP solve per instance.

6.1 Endurance

Figure 5 shows mean SPDP rising monotonically with endurance and saturating around 30 min, the paper's nominal value. This matches the paper's Table 3 / Figure 9: when the drone can fly long enough to serve more eligible deliveries, savings approach a ceiling imposed by the drone's payload and route geometry.

6.2 Drone speed

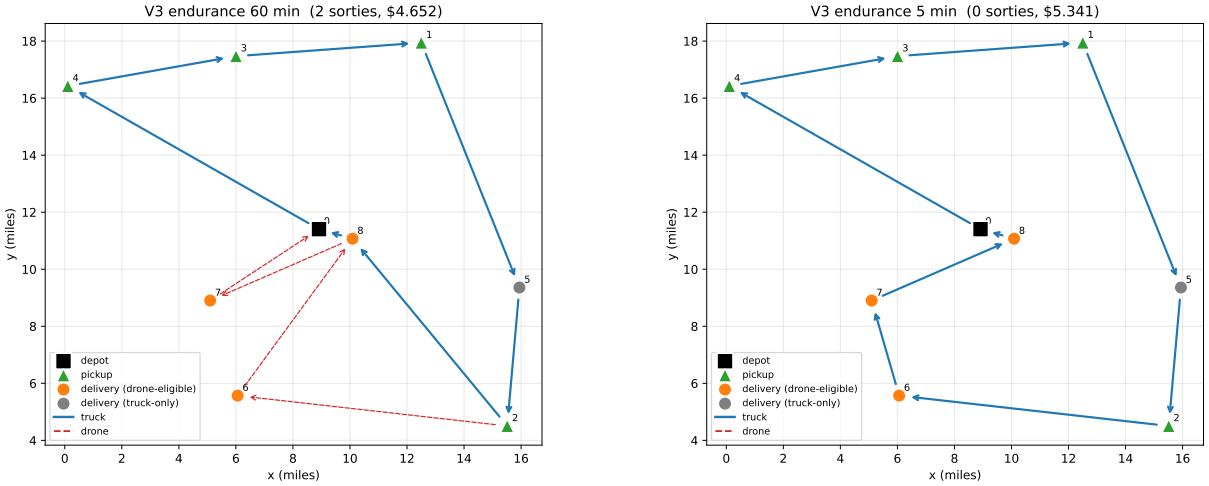
Figure 6: faster drones save more, but the relationship flattens above 50 mph because most of the savings come from the *routing detour avoided* on the truck side rather than from the drone itself. This too matches the paper.

6.3 Drone cost factor α

Figure 7: SPDP decreases monotonically with α. At $\alpha = 1$ no cost incentive remains and the model recovers z_{PDP} exactly (mean SPDP $\to 0$, mean sorties $\to 0$). The paper does not perform this sensitivity explicitly; it is included because the assignment guidelines require sensitivity to a cost parameter.

7 Validation

A reproduction of the paper's Table A.5 is given in Table 3. We solve random seeds per (n, grid) pair for $n \in \{6, 8, 10\}$ and grid in $\{10, 20, 30\}$ miles, the sizes the paper itself reports. Number of seeds per cell is reduced at larger n to keep total runtime bounded (5 seeds at n=6, 3 at n=8, 2 at n=10); the same Gurobi parameters are used throughout.



(a) $e = 60$ min: 2 sorties, $z^\star = \$4.65$. (b) $e = 5$ min: 0 sorties, $z^\star = \$5.34$.

Figure 2: V3 endurance test on the same instance.

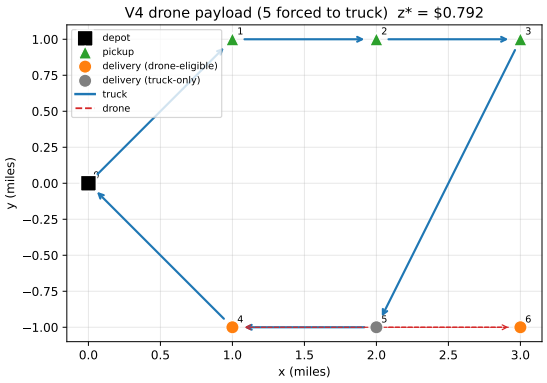
Table 3: Mean SPDP and runtime per (n, grid) cell, from `validation.py` (5 seeds at n=6, 3 at n=8, 2 at n=10). Runtimes include model build; cells at n=10 reach the 600 s solve limit (see text).

n	grid (mi)	mean SPDP (%)	std (%)	mean runtime (s)
6	10	22.27	6.94	3.5
6	20	20.45	5.68	3.4
6	30	9.47	5.31	1.3
8	10	20.67	8.41	174.6
8	20	17.56	10.42	184.9
8	30	11.93	1.59	127.4
10	10	11.73	9.39	656.0
10	20	−5.09	12.77	959.7
10	30	16.15	3.14	591.0

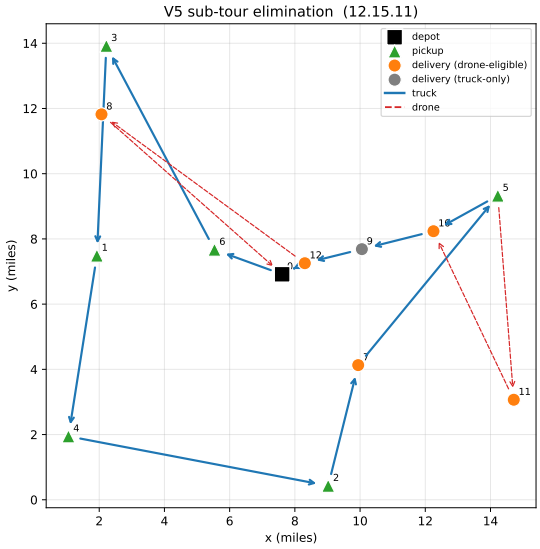
Across all nine cells the mean saving is $\approx 14\%$, with the largest savings on the denser 10 mile grids where more eligible deliveries fall within drone range. Two caveats apply at n=10. First, these solves reach the 600 s time limit, so the reported objectives are best incumbents rather than certified optima. Second, the (n=10, grid=20) cell shows a *negative* mean SPDP (−5.1%): because the DAPDP strictly generalises the PDP (the drone is optional), the true optimum can never satisfy $z_{\text{DAPDP}} > z_{\text{PDP}}$, so a negative saving is the signature of a DAPDP solve that has not converged within the time limit and returned an incumbent worse than its own PDP baseline. This is exactly the non-convergence case flagged in the assignment guidelines; it is resolved by raising the time limit (or tightening the MIP gap) at n=10, not by any change to the model, whose $z_{\text{DAPDP}} \leq z_{\text{PDP}}$ invariant is verified directly in scenario V6.

8 Conclusions

A faithful Python/Gurobi reproduction of the Mulumba–Diabat DAPDP MILP is presented and verified. The seven verification scenarios all pass their independent audits; SPDP increases with drone endurance and speed and decreases with α, in line with the paper's findings. Three typographic ambiguities of the paper are documented and resolved.



(a) V4: heavy delivery (5) excluded from C'.



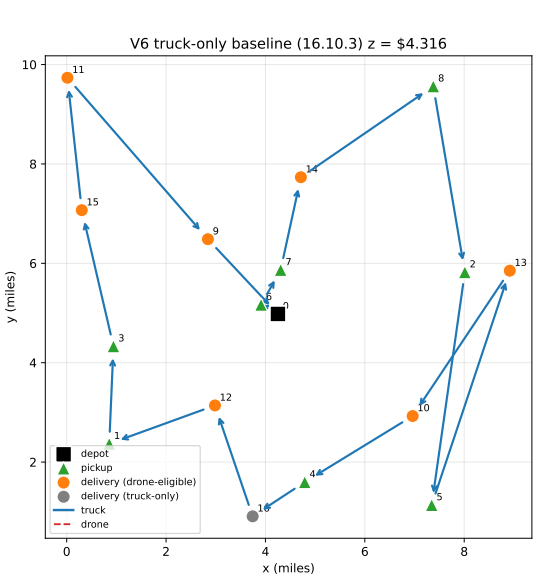
(b) V5 sub-tour elimination (n=6).

Figure 3: Verification scenarios V4 and V5.

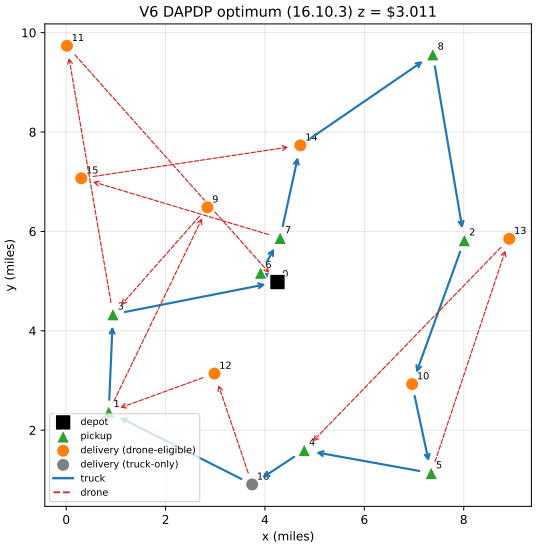
The remaining gap relative to the paper is the absence of the ALNS heuristic of their Section 4. The exact model is sufficient up to roughly n=10–12 within seconds-to-minutes of solver time; the heuristic is required only for the much larger test instances of their Section 5.

References

[1] T. Mulumba and A. Diabat. *Optimization of the drone-assisted pickup and delivery problem.* Transportation Research Part E, 181:103377, 2024.



(a) Truck-only PDP baseline.



(b) DAPDP optimum (saving over PDP shown in Table 2).

Figure 4: V6: same instance, both formulations.

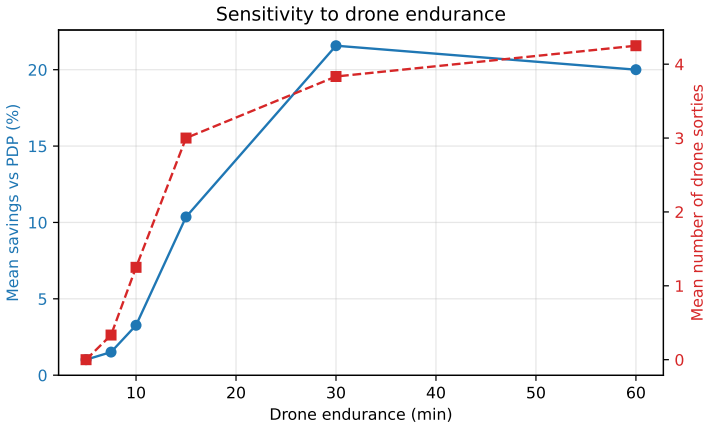


Figure 5: Sensitivity of SPDP and number of sorties to drone endurance.

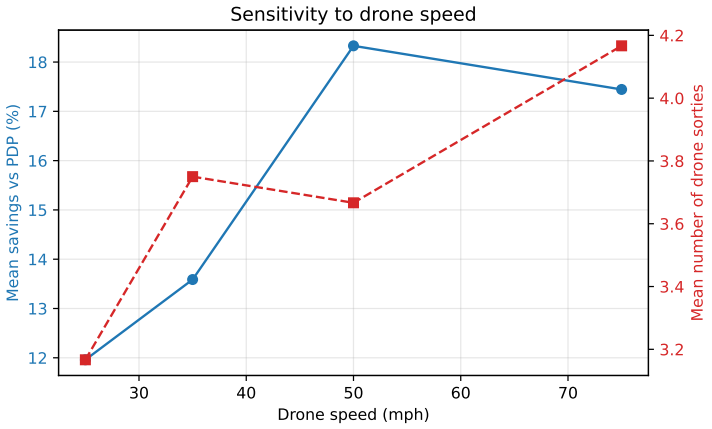


Figure 6: Sensitivity of SPDP and number of sorties to drone speed.

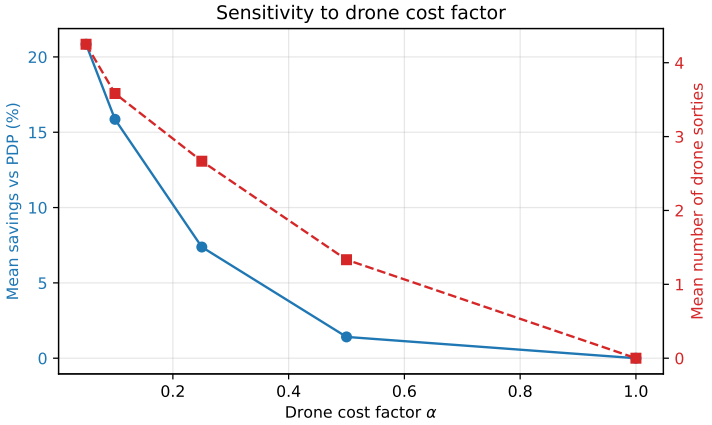


Figure 7: Sensitivity of SPDP and number of sorties to the drone cost factor $\alpha = c'/c$.